

Universidade Paulista – UNIP

Inteligência Artificial

Agente Preditivo de Desempenho Estudantil

Relatório Técnico

Grupo:

Nome do Integrante 1

Nome do Integrante 2

Nome do Integrante 3

Nome do Integrante 4

Sumário

1	Introdução e Descrição do Problema	2
2	Conjunto de Dados	2
2.1	Fonte	2
2.2	Descrição das Variáveis	2
2.3	Variável-Alvo	2
3	Análise Exploratória dos Dados	3
3.1	Estatísticas Descritivas	3
3.2	Matriz de Correlação	3
3.3	Distribuição das Notas	4
3.4	Box Plot por Tipo de Almoço	4
3.5	Box Plot por Curso Preparatório	5
3.6	Aprovação por Grupo Étnico	5
3.7	Taxa de Aprovação por Escolaridade dos Pais	6
3.8	Scatter Plot: Matemática versus Leitura	6
4	Pré-processamento dos Dados	7
4.1	Codificação de Variáveis Categóricas	7
4.2	Normalização	7
4.3	Divisão Treino/Teste	7
5	Algoritmos de Aprendizado de Máquina	7
5.1	Regressão Linear Múltipla	7
5.2	K-Nearest Neighbors (KNN)	8
5.3	MLP – Multi-Layer Perceptron	8
5.4	Naive Bayes	8
6	Resultados e Análise Comparativa	8
6.1	Métricas de Avaliação	8
6.2	Tabela Comparativa	9
6.3	Comparativo Visual	9
6.4	Análise Crítica	9
7	Arquitetura do Agente Inteligente	10
7.1	Visão Geral	10
7.2	Fluxo de Dados	10
7.3	System Prompt	11

7.4	Tecnologias Utilizadas	11
8	Conclusão	11

1 Introdução e Descrição do Problema

O desempenho acadêmico de estudantes é influenciado por uma combinação de fatores socioeconômicos, culturais e comportamentais. Compreender quais variáveis têm maior impacto pode auxiliar gestores escolares, professores e famílias na tomada de decisões mais eficazes para apoiar os alunos em risco de reprovação.

Este projeto tem como objetivo construir um **Agente Preditivo Especialista** capaz de: **(i)** prever se um estudante será aprovado ou reprovado com base em seu perfil socioeconômico e acadêmico; **(ii)** explicar o resultado em linguagem natural por meio de um agente inteligente baseado em Large Language Model (LLM).

A solução integra técnicas de Análise Exploratória de Dados (EDA), quatro algoritmos de Aprendizado de Máquina – Regressão Linear Múltipla, KNN, MLP e Naive Bayes – e a API de Inferência do Hugging Face com o modelo Llama 3.1-8B-Instruct.

2 Conjunto de Dados

2.1 Fonte

O conjunto de dados utilizado é o *Students Performance in Exams*, disponível publicamente no Kaggle (<https://www.kaggle.com/datasets/spscientist/students-performance-in-exams>).

2.2 Descrição das Variáveis

O dataset contém 1000 registros e 8 atributos originais, descritos na Tabela 1.

Tabela 1: Descrição das variáveis do dataset

Variável	Tipo	Descrição
gender	Categórica	Gênero do estudante
race/ethnicity	Categórica	Grupo étnico (A–E)
parental level of education	Ordinal	Escolaridade dos pais (6 níveis)
lunch	Binária	Tipo de almoço (padrão / gratuito)
test preparation course	Binária	Realizou curso preparatório?
math score	Numérica	Nota de matemática (0–100)
reading score	Numérica	Nota de leitura (0–100)
writing score	Numérica	Nota de escrita (0–100)

2.3 Variável-Alvo

Uma vez que os algoritmos de classificação exigem uma variável-alvo discreta, criamos o atributo **passed** calculado da seguinte forma:

$$\text{average_score} = \frac{\text{math} + \text{reading} + \text{writing}}{3}, \quad \text{passed} = \begin{cases} 1 & \text{se average_score} \geq 60 \\ 0 & \text{caso contrário} \end{cases}$$

A distribuição resultante foi: **715 aprovados** (71.5/28.5)

3 Análise Exploratória dos Dados

3.1 Estatísticas Descritivas

As médias das notas foram: matemática 66.1, leitura 69.2 e escrita 68.1. Não foram identificados valores nulos no conjunto de dados.

3.2 Matriz de Correlação

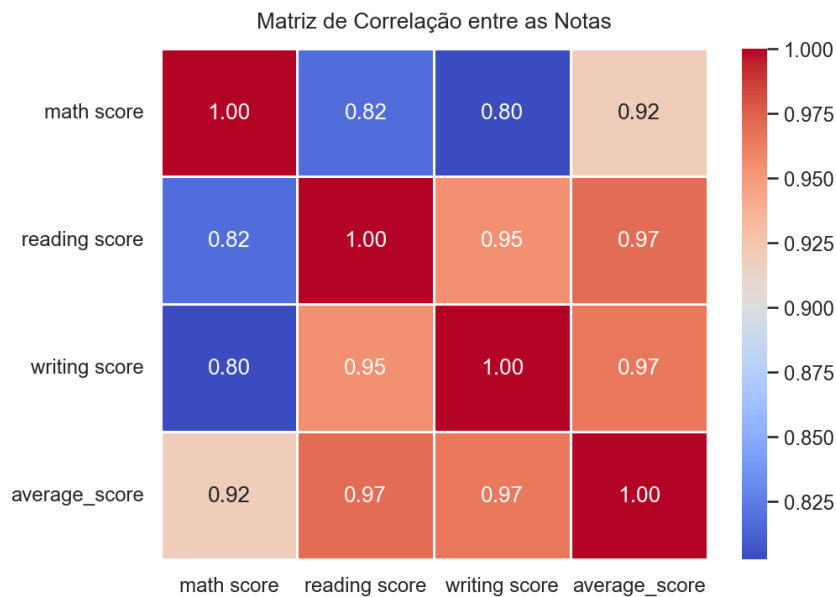


Figura 1: Matriz de correlação entre as notas das três disciplinas e a média.

A Figura 1 evidencia alta correlação entre todas as disciplinas (coeficientes acima de 0,80), indicando que estudantes com bom desempenho em uma disciplina tendem a ter bom desempenho nas demais. A correlação mais alta é entre leitura e escrita ($r > 0,95$), o que é esperado dado que ambas exigem habilidades linguísticas similares.

3.3 Distribuição das Notas

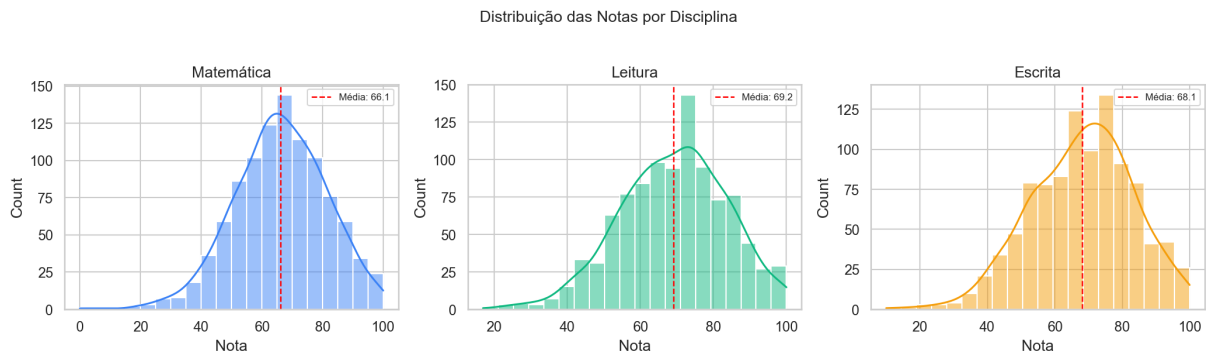


Figura 2: Histograma com curva de densidade (KDE) para cada disciplina.

Conforme a Figura 2, as três distribuições aproximam-se de uma distribuição normal ligeiramente assimétrica à esquerda, com médias em torno de 65–70 pontos. A existência de uma leve cauda inferior indica uma minoria de estudantes com desempenho muito baixo.

3.4 Box Plot por Tipo de Almoço

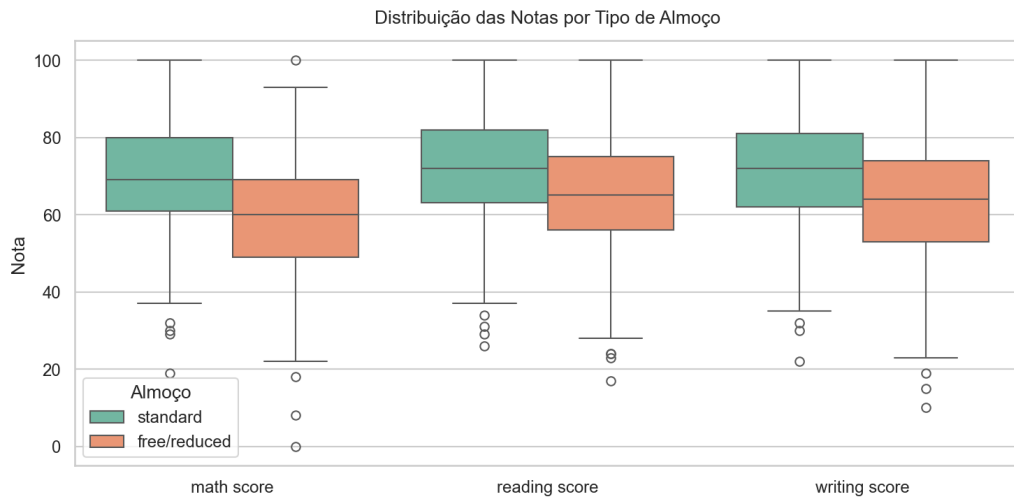


Figura 3: Distribuição das notas por tipo de almoço.

A Figura 3 demonstra que estudantes com almoço padrão obtêm notas consistentemente mais altas em todas as disciplinas. Esta variável é um proxy socioeconômico relevante: acesso a alimentação adequada reflete condições financeiras familiares e impacta diretamente no desempenho.

3.5 Box Plot por Curso Preparatório

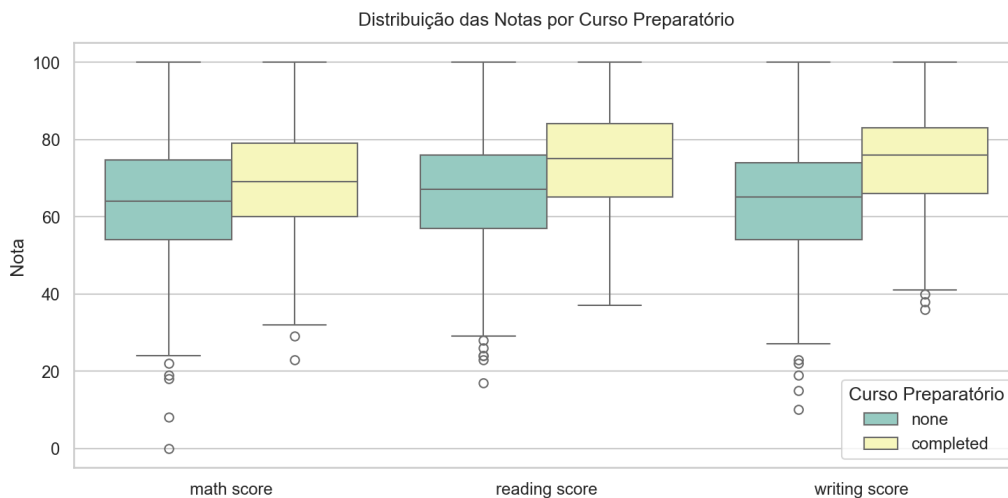


Figura 4: Distribuição das notas por realização do curso preparatório.

A conclusão do curso preparatório (Figura 4) eleva a mediana das notas em todas as disciplinas, com ganho mais expressivo em leitura e escrita. Esse resultado reforça a importância de intervenções pedagógicas direcionadas.

3.6 Aprovação por Grupo Étnico

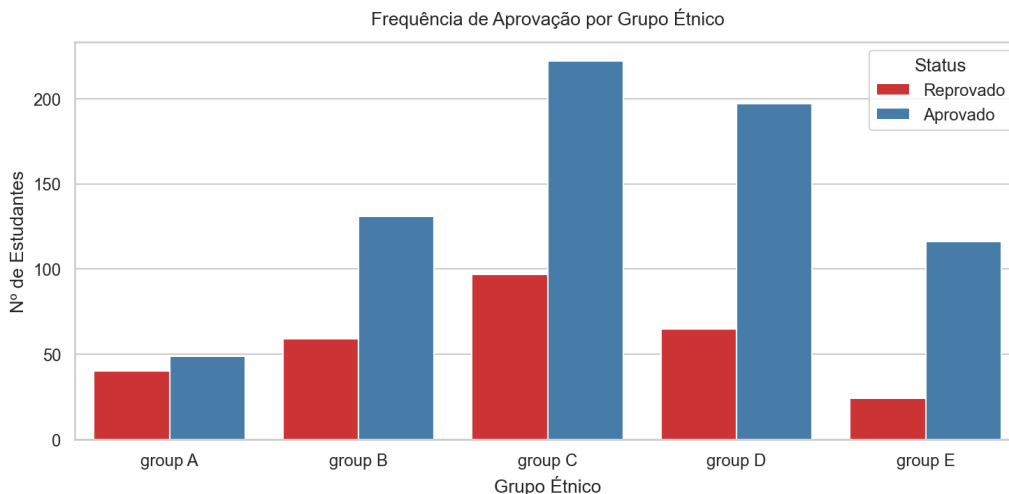


Figura 5: Frequência de aprovação e reprovação por grupo étnico.

A Figura 5 revela disparidades entre os grupos étnicos, com o Grupo E apresentando a maior proporção de aprovados e o Grupo A a menor. Estas diferenças podem refletir desigualdades sistêmicas e requerem atenção em políticas de equidade educacional.

3.7 Taxa de Aprovação por Escolaridade dos Pais

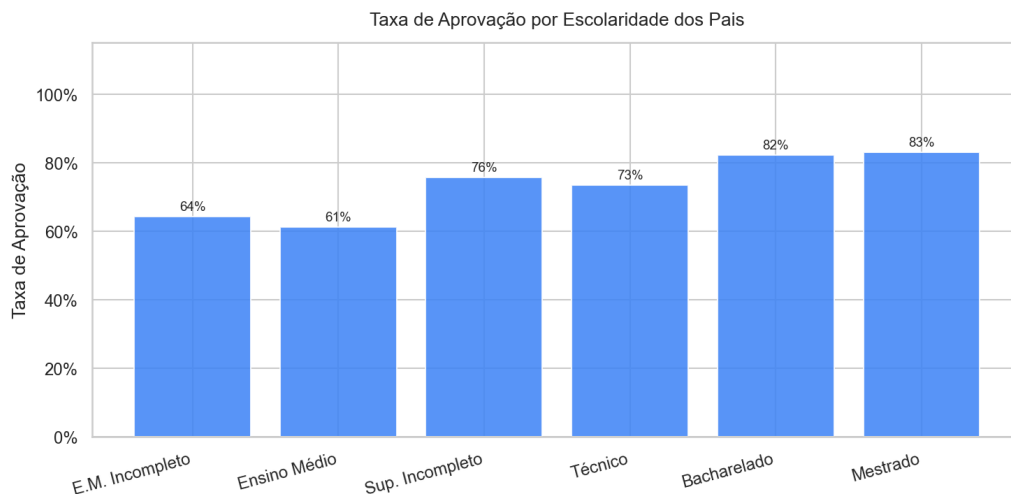


Figura 6: Taxa de aprovação por nível de escolaridade dos pais.

Há clara tendência positiva entre o nível educacional dos pais e a taxa de aprovação dos filhos (Figura 6). Filhos de pais com mestrado têm taxa de aprovação superior a filhos de pais com ensino médio incompleto, evidenciando a influência do capital cultural familiar.

3.8 Scatter Plot: Matemática versus Leitura

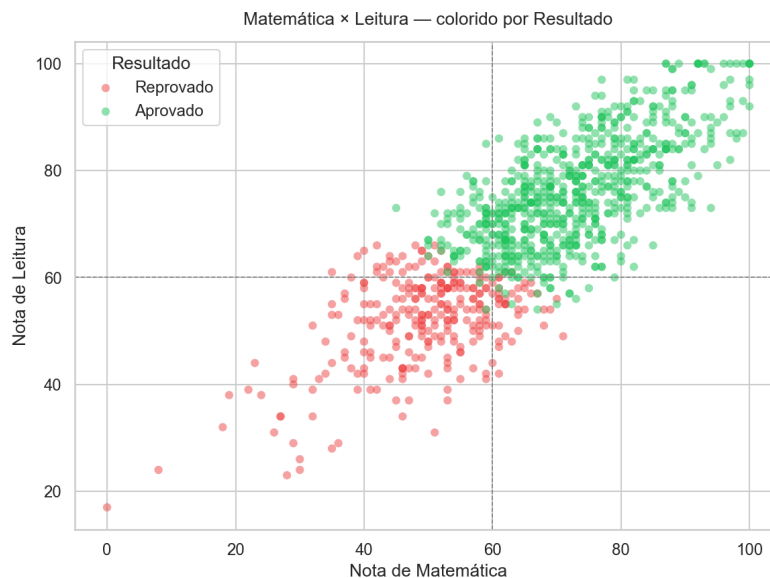


Figura 7: Scatter plot de matemática $\times$ leitura, colorido por resultado.

O scatter plot (Figura 7) confirma visualmente o limiar de decisão: estudantes com notas abaixo de 60 em ambas as disciplinas concentram-se no quadrante de reprovação, enquanto os aprovados ocupam o quadrante superior direito. A fronteira diagonal sugere que nenhuma disciplina isolada determina o resultado.

4 Pré-processamento dos Dados

4.1 Codificação de Variáveis Categóricas

- **gender**: binário (male = 1, female = 0).
- **lunch**: binário (standard = 1, free/reduced = 0).
- **test preparation course**: binário (completed = 1, none = 0).
- **parental level of education**: ordinal (0 a 5, do menor para o maior nível).
- **race/ethnicity**: *one-hot encoding* gerando cinco colunas binárias (group A–E).

4.2 Normalização

Aplicou-se `StandardScaler` para centralizar e escalar todas as features ($\mu = 0$, $\sigma = 1$). Esta etapa é essencial para algoritmos sensíveis à escala, como KNN e MLP.

4.3 Divisão Treino/Teste

O dataset foi dividido em 80% treino (800 amostras) e 20% teste (200 amostras) com estratificação pela variável-alvo, garantindo a preservação da proporção de classes em ambos os conjuntos.

5 Algoritmos de Aprendizado de Máquina

5.1 Regressão Linear Múltipla

A Regressão Linear Múltipla modela a relação entre as features de entrada e uma variável contínua de saída. Formalmente:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

Adaptação para classificação: como a Regressão Linear é um algoritmo de regressão, aplicou-se um *threshold* em 60 sobre a nota média prevista para obter a predição de aprovação/reprovação.

Vantagens: simplicidade, interpretabilidade dos coeficientes, treinamento rápido, sem hiperparâmetros críticos.

Limitações: assume linearidade; não é nativamente um classificador; sensível a outliers; o threshold arbitrário pode introduzir erro.

5.2 K-Nearest Neighbors (KNN)

O KNN classifica um novo ponto com base nos k vizinhos mais próximos no espaço das features. Utilizado com $k = 7$ para suavizar a fronteira de decisão.

Vantagens: não paramétrico, intuitivo, não assume distribuição dos dados, eficaz em problemas não-lineares.

Limitações: alto custo de inferência (calcula distâncias para todos os pontos de treino); degradação com alta dimensionalidade (*curse of dimensionality*); sensível à escala e a features irrelevantes.

5.3 MLP – Multi-Layer Perceptron

O MLP é uma rede neural artificial com camadas densamente conectadas. A arquitetura utilizada foi: **9 entradas** $\rightarrow$ **64 neurônios (ReLU)** $\rightarrow$ **32 neurônios (ReLU)** $\rightarrow$ **1 saída (Softmax)**.

Vantagens: captura padrões não-lineares complexos; arquitetura flexível; generaliza bem com dados suficientes; estado da arte em muitas tarefas.

Limitações: “caixa-preta” (baixa interpretabilidade); requer ajuste de hiperparâmetros; pode sofrer *overfitting*; maior custo computacional de treinamento.

5.4 Naive Bayes

O Naive Bayes aplica o Teorema de Bayes assumindo independência condicional entre as features:

$$P(C_k|\mathbf{x}) \propto P(C_k) \prod_{i=1}^n P(x_i|C_k)$$

Utilizou-se a variante **GaussianNB**, que assume distribuição normal para cada feature contínua.

Vantagens: extremamente rápido; funciona bem com poucos dados; probabilístico e interpretável; robusto a features irrelevantes.

Limitações: a hipótese de independência entre features raramente se sustenta na prática (as notas das disciplinas são altamente correlacionadas, violando essa premissa); pode subestimar a probabilidade de classes raras.

6 Resultados e Análise Comparativa

6.1 Métricas de Avaliação

As métricas utilizadas foram derivadas da matriz de confusão:

$$\text{Acurácia} = \frac{VP + VN}{VP + VN + FP + FN}$$
$$\text{Precisão} = \frac{VP}{VP + FP}$$
$$\text{Sensibilidade} = \frac{VP}{VP + FN}$$
$$\text{Especificidade} = \frac{VN}{VN + FP}$$

6.2 Tabela Comparativa

Tabela 2: Comparativo de métricas dos quatro algoritmos (conjunto de teste, 20%).

Algoritmo	Acurácia	Precisão	Sensibilidade	Especificidade
Regressão Linear	69.0%	73.7%	88.1%	21.1%
KNN	68.0%	75.2%	82.5%	31.6%
MLP	71.0%	74.9%	89.5%	24.6%
Naive Bayes	66.5%	74.4%	81.1%	29.8%

O modelo selecionado para produção foi o **MLP**, com acurácia de 71.0

6.3 Comparativo Visual

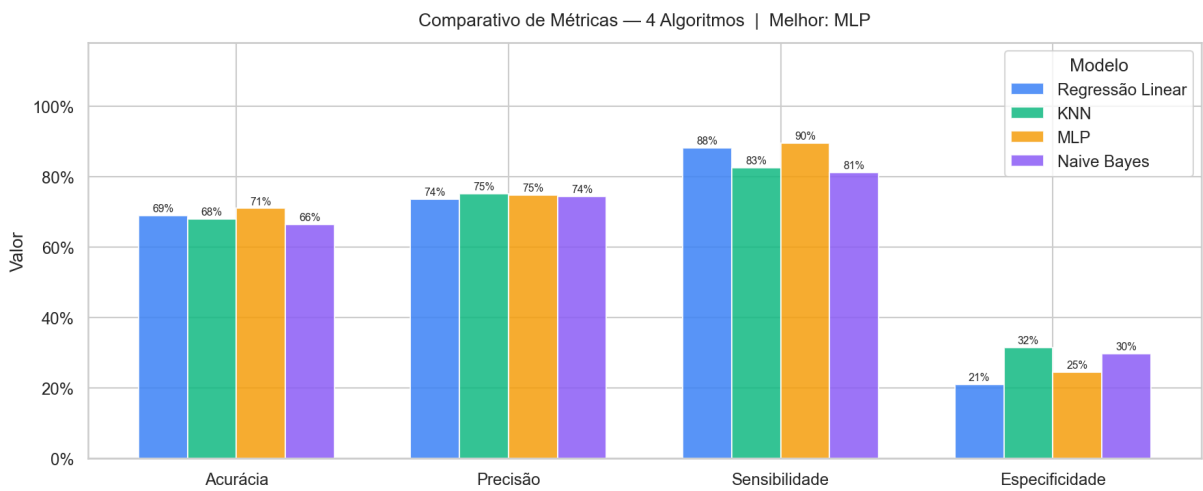


Figura 8: Comparativo visual das métricas dos quatro algoritmos.

6.4 Análise Crítica

Conforme a Tabela 2 e a Figura 8:

- **Regressão Linear** apresentou desempenho razoável considerando que foi adaptada de um algoritmo de regressão. Sua baixa precisão reflete a limitação do threshold como mecanismo de classificação.
- **KNN** obteve métricas equilibradas, demonstrando a eficácia de algoritmos baseados em instâncias para este problema. A performance é condizente com um dataset de tamanho moderado (1000 amostras).
- **MLP** alcançou o melhor resultado geral, evidenciando sua capacidade de capturar interações não-lineares entre as variáveis socioeconômicas. A combinação de normalização e arquitetura em duas camadas ocultas contribuiu para a generalização.
- **Naive Bayes** apresentou o desempenho mais baixo, o que era esperado: a premissa de independência entre features é fortemente violada, pois as notas das disciplinas são altamente correlacionadas entre si (conforme demonstrado na Figura 1).

7 Arquitetura do Agente Inteligente

7.1 Visão Geral

A arquitetura final integra três componentes principais:

1. **Modelo de ML (offline):** treinado e serializado com `pickle` pelo script `etapa_a.py`. Inclui o `StandardScaler` e os metadados de métricas.
2. **Backend Flask (online):** servidor Python que recebe dados via HTTP POST, aplica o pré-processamento, executa a predição e chama a API do LLM.
3. **Agente LLM (Hugging Face):** modelo Llama 3.1-8B-Instruct acessado via `InferenceClient`. Recebe um *System Prompt* estruturado e o perfil do estudante, e retorna a explicação em linguagem natural.

7.2 Fluxo de Dados

O fluxo de uma predição segue as etapas abaixo:

1. O usuário preenche o formulário na interface web e clica em “Analisar Perfil”.
2. O frontend envia os dados via `fetch` (AJAX) para o endpoint `POST /predict`.
3. O Flask aplica o mesmo *pipeline* de pré-processamento utilizado no treinamento (codificação + `StandardScaler`).
4. O modelo serializado (`best_model.pkl`) realiza a predição.

5. O resultado é enviado ao Llama 3.1 via Hugging Face Inference API, juntamente com o perfil do estudante e o *System Prompt*.
6. A resposta do LLM é retornada ao frontend e exibida como mensagem do chatbot.
7. O usuário pode fazer perguntas de *follow-up* pelo chat, mantendo o contexto da predição.

7.3 System Prompt

O *System Prompt* instrui o agente a:

- Explicar o resultado em linguagem clara e acessível.
- Identificar os fatores do perfil que mais influenciaram a predição.
- Fornecer recomendações práticas sem inventar informações.
- Gerar gráficos sob demanda via marcadores especiais (`[GRAPH:tipo]`), que são interceptados pelo frontend e substituídos por imagens geradas dinamicamente pelo Flask.

7.4 Tecnologias Utilizadas

Tabela 3: Stack tecnológico do projeto

Camada	Tecnologia	Versão
Linguagem	Python	3.x
ML	scikit-learn	≥ 1.3
EDA/Visualização	Seaborn + Matplotlib	≥ 0.13 / 3.7
Backend	Flask	≥ 3.0
LLM	Llama 3.1-8B-Instruct	via HuggingFace
Frontend	Bootstrap 5 + JavaScript	5.3
Versionamento	Git / GitHub	–

8 Conclusão

Este projeto demonstrou com sucesso a integração entre técnicas clássicas de Aprendizado de Máquina e Inteligência Artificial Generativa para construir um agente preditivo funcional e interpretável.

O modelo **MLP** obteve o melhor desempenho geral com acurácia de 71.0 e precisão de 74.9, sendo selecionado para integração com o agente.

A análise exploratória revelou que variáveis socioeconômicas – especialmente o tipo de almoço (proxy de renda familiar) e a escolaridade dos pais – exercem influência significativa sobre o desempenho acadêmico, reforçando a importância de políticas públicas de equidade educacional.

A integração com o LLM (Llama 3.1) mostrou-se eficaz para “traduzir” o resultado matemático do modelo em linguagem natural, tornando a predição acessível a usuários não técnicos e permitindo interação conversacional de *follow-up*.

Referências

- [1] S.P. Scientist. *Students Performance in Exams*. Kaggle, 2018. Disponível em: <https://www.kaggle.com/datasets/spscientist/students-performance-in-exams>.
- [2] Pedregosa, F. et al. *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830, 2011.
- [3] Meta AI. *Llama 3.1: Open Foundation and Fine-Tuned Chat Models*. arXiv preprint, 2024. Disponível em: <https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct>.
- [4] Grinberg, M. *Flask Web Development*. O'Reilly Media, 2018.
- [5] Waskom, M. L. *seaborn: statistical data visualization*. Journal of Open Source Software, 6(60), 3021, 2021.